

WEEK 09 · STATE, RENDER, ONE SOURCE OF TRUTH

ONE PLACE WHERE THINGS ARE TRUE

Last week the page started answering. Today it stops holding opinions of its own.

WHERE WE ARE

An application that works, and a bug in it that cannot be fixed by fixing it.

AFTER TODAY YOU CAN

Take any list-shaped application apart into three layers and say what each one is forbidden to do.

WHAT THIS UNLOCKS

Every project in this room ends up the same shape — which is the only reason the week-13 exam can be the same for all of you.

**This is the week the course
names its architecture.**

A BUG YOU CANNOT FIX BY FIXING IT

WHAT HAPPENS

Remove an item. Add a different one. **The removed one is back.**

THE OBVIOUS FIX

The remove handler deletes the row and forgets the array. So make it delete from the array too.

WHY THAT IS NOT A FIX

Now **two** things have to be updated on every removal, by whoever writes the next handler. You have made the bug harder to hit, not impossible.

**Every bug this week is the same bug:
something is true in two places.**

TWO COPIES, AND THE DISTANCE BETWEEN THEM

left: the array · right: the rows · far right: how far apart, computed↻ reloadopen ↗

ONE LIST, TWO PLACES IT IS WRITTEN DOWN

push to the array only

append a row only

remove a row only

compare the two

redraw from the array

reset

THE ARRAY — ITEMS

```
[  
  "flour",  
  "olive oil",  
  "almond milk"  
]
```

THE SCREEN — #LIST

flour

olive oil

almond milk

HOW FAR APART THEY ARE

0

the two agree

Nothing on this page repairs the drift, and that is the point: every repair you can think of is one more line that one more handler has to remember, forever.

► **PRESS "APPEND A ROW ONLY" THREE TIMES, THEN ASK WHAT THE ARRAY HOLDS.** Vote before you press compare.

FOUR WAYS THE SAME DISEASE SHOWS UP IN YOUR OWN CODE

- 1 Remove an item, add one —
the removed one returns.
- 2 Search, then add something that does not match —
it appears anyway.
- 3 Search, press +1 on a row, clear the search —
a different row went up.
- 4 Sort, then add —
the sort is gone.

► All four are in the file you get in ninety minutes. All four need **TWO ACTIONS IN THE RIGHT ORDER**, which is why nobody found them while writing it.

THERE ARE EXACTLY TWO WAYS TO KEEP A SCREEN UP TO DATE

PATCH IT

Each handler makes the change the user asked for, in the DOM, and keeps the data in step by hand.

Correct for any one action. Not correct for two.

REDRAW IT

Each handler changes the data. One function draws the whole screen from the data, every time.

Correct for any sequence, because there is no sequence.

Not a trick — giving up an optimisation you were never paid for.

THE SAME APPLICATION, TWICE

left: patched · right: redrawn · identical data, identical buttons

↻ reload open ↗

THE SAME APPLICATION, PATCHED ON THE LEFT AND REDRAWN ON THE RIGHT

add one remove the first +1 on the first sort by quantity 20 random operations, both panels redraw both from their data reset

PATCHED — EVERY HANDLER DRAWS

flour	2
olive oil	1
almond milk	4
cheese	1

30 lines across four handlers

agrees with its data

REDRAWN — EVERY HANDLER ONLY CHANGES DATA

flour	2
olive oil	1
almond milk	4
cheese	1

20 lines across four handlers

agrees with its data

The left panel is not written badly. Every handler in it is correct on its own, and it was tested one button at a time. What it cannot survive is the two hundred orderings of those buttons that nobody tested.

► Press each button once and both agree. Then **20 RANDOM OPERATIONS** — and then “redraw both”.

6

READ THE TWO REMOVE HANDLERS, AND NOTHING ELSE

PATCHED

```
1 function onClick(e) {  
2   const row = e.target.closest('li');  
3   row.remove();  
4   count.textContent = rows();  
5   empty.hidden = rows() > 0;  
6 }
```

Three facts to remember. It forgot the only one that mattered.



REDRAWN

```
1 function onClick(e) {  
2   const row = e.target.closest('li[data-id]');  
3   const keep = (i) => i.id !== row.dataset.id;  
4   setState({ items: items.filter(keep) });  
5 }
```

One fact. There is nothing left to forget.



THE ONE-WAY LOOP

```
1  state → render(state) → the screen
2  ↑
3  |_____ events.js ←
```

Data flows one way round. The screen never writes back.

**This has a name in this course,
and we will use it for the rest of
the semester: the one-way loop.**

THREE LAWS, AND EVERYTHING TODAY FOLLOWS FROM THEM

1 · ONE TRUTH

If it is on screen, it came from `state`. If it is not in `state`, it is not real.

2 · ONE PAINTER

Only `render(state)` writes to the screen. Nothing else, ever.

3 · ONE DIRECTION

Handlers change `state`. Nothing reads the screen to find out what is true.

► A fourth follows from all three and it is the one people lose marks on: **ANYTHING DERIVABLE IS DERIVED.**

“SO THIS IS REACT?”

WHAT REACT ADDS

A description of the screen you return instead of draw, and a diff so that redrawing everything only **writes** what changed.

WHAT REACT DOES NOT ADD

The model. **One state, one render, one direction** is the idea React was built to make cheap — it did not invent it.

WHY WE DO IT BY HAND

Because the part that is hard to learn is the model, and the part that is easy to learn is the library. Doing it once by hand makes the second one a Tuesday.

You are learning React thinking. The import statement is next semester's problem.

WHAT EACH LAYER IS FORBIDDEN TO DO

FILE	ITS JOB	WHAT IT MAY NEVER DO
<code>js/state.js</code>	the object, and every derivation of it	touch the DOM
<code>js/render.js</code>	state in, screen out	change state · add listeners · read the screen
<code>js/events.js</code>	turn a user action into a change of state	draw anything
<code>js/app.js</code>	wire the three together, once	grow

► The third column is the useful one. **LAYERS ARE DEFINED BY THEIR PROHIBITIONS**, not by their contents.

THE WHOLE APPLICATION, IN FOUR LINES

```
1 seed(SEED_ITEMS);           // data  -> state
2 subscribe(render);           // state -> screen, on every change, forever
3 wire();                       // screen -> state, via handlers that only setState
4 render(getState());           // draw once, because subscribing does not fire
```

Line 4 is the one people forget, and the page it produces looks exactly like a broken render function.

WHAT BELONGS IN THE OBJECT, AND WHAT DOES NOT

IN

```
1 const state = {  
2   items: [],           // not computable  
3   query: '',           // the user typed  
4   category: 'all',     // the user chose  
5   sort: 'title',       // the user picked  
6   undo: null,          // a past state  
7 };
```

OUT

```
1 const state = {  
2   visibleItems: [],    // = filter + sort  
3   count: 0,            // = its length  
4   categories: [],      // = the shelves  
5   summary: {},         // = page three  
6 };
```

► The test is one question per field: **CAN I COMPUTE IT FROM THE OTHERS?** If yes, it does not live here.

A FUNCTION, NOT AN OBJECT LITERAL

SHARED BY ACCIDENT

```
1 export const INITIAL = {  
2   items: [],  
3   query: '',  
4 };
```

One object. Every page that imports it gets **the same one**, and one mutation leaks into the next.



FRESH EVERY TIME

```
1 function initialState() {  
2   return {  
3     items: [],  
4     query: '',  
5   };  
6 }
```

Called once per load. **Week 7's value-versus-reference**, arriving where it costs something.



ONE DOOR IN, AND IT IS THE REASON EVERYTHING ELSE WORKS

```
1 let state = initialState();
2 const listeners = new Set();
3
4 export function setState(patch) {
5   state = { ...state, ...patch };
6   for (const listener of listeners) listener(state);
7 }
8
9 export const getState = () => state;
```

► Because **NOTHING OUTSIDE THIS FILE ASSIGNS TO STATE**, “did anybody change the collection without redrawing?” has a permanent answer of no.

LIVE CODING · 6 MINUTES

TAKE THE REMOVE HANDLER OUT OF THE MONOLITH

- 1 Open the week-8 `app.js` on the projector.
Do not read it first.
- 2 Create `js/state.js` with `getState`, `setState`, `subscribe`. Move `items` into it.
- 3 Move the remove handler into `js/events.js`, and delete every line in it that touches an element.
- 4 **Run it.**
The page is blank. Ask the room why before you fix it.

► Then add line 4 of `app.js`, and it works.

RENDER(STATE) IS A FUNCTION OF ITS ARGUMENT

edit the object on the left, press render, and there is no third input

reload

open

THE SCREEN IS A FUNCTION OF THIS OBJECT, AND OF NOTHING ELSE

render(state)

render twice, compare the two

delete a row straight from the DOM

subscribe without the first draw

reset

STATE — EDIT IT

the whole application, as one object

```
{
  "items": [
    {
      "id": "a1",
      "name": "flour",
      "qty": 2
    },
    {
      "id": "a2",
      "name": "olive oil",

```

valid JSON

THE SCREEN — RENDER(STATE)

showing 3 of 3 · renders so far: 1

almond milk

4

flour

2

olive oil

1

nothing measured yet

There is no button here that changes the screen without changing the object. That is not a limitation of the demo — it is the whole rule, and everything else this week follows from it.

► **“DELETE A ROW STRAIGHT FROM THE DOM” — IS THE ITEM GONE?** Take the vote. Then press render.

THREE THINGS RENDER MAY NEVER DO, AND WHY EACH ONE BITES

FORBIDDEN

WHAT HAPPENS IF YOU DO

change state

it calls itself. You debug a frozen tab.

add a listener

one more listener per redraw. After ten keystrokes the handler runs ten times.

read the DOM to decide

the screen is now a source of truth, and law 1 is gone.

► The second one is invisible everywhere except the Event Listeners panel — and in how many times your `console.log` prints.

A COMPONENT IS A FUNCTION: ITEM IN, ELEMENT OUT

```
1  const badge = (text) => `<span class="qty">${esc(text)}</span>`;
2
3  const row = (item) => `<li data-id="${esc(item.id)}">
4    <span class="item-title">${esc(item.title)}</span>
5    ${badge(item.number)}
6  </li>`;
7
8  const list = (items) => `<ul id="list">${items.map(row).join('')}</ul>`;
```

► No side effects, no knowledge of where the result will be put, and `list` calls `row` calls `badge`. That is composition, with no framework in the room.

THE PRICE OF A TEMPLATE LITERAL IS SIX LINES

the same row four ways · identical markup · and one item with a hostile name

reload

open

A COMPONENT IS A FUNCTION: ITEM IN, MARKUP OUT

build the same row four ways

compose: list calls row calls badge

give one item a hostile name

reset

1 · STRING CONCATENATION

```
function concatRow(item) {
  return (
    '<li data-id="' +
    item.id +
    '>' +
    '<span class="name">' +
    item.name +
    '</span>' +
    '<span class="qty">' +
    item.qty +
    '</span>' +
    '</li>'
  );
}
```

flour

2

2 · TEMPLATE LITERAL, UNESCAPED

```
function rawRow(item) {
  return `<li data-id="${item.id}">
  <span class="name">${item.name}</span>
  <span class="qty">${item.qty}</span>
</li>`;
}
```

flour

2

3 · TEMPLATE LITERAL, ESCAPED

```
function safeRow(item) {
  return `<li data-id="${esc(item.id)}">
  <span class="name">${esc(item.name)}</span>
  <span class="qty">${esc(item.qty)}</span>
</li>`;
}
```

flour

2

4 · CREEATEELEMENT

```
function domRow(item) {
  const li = document.createElement('li');
  li.dataset.id = item.id;
  const name = document.createElement('span');
  name.className = 'name';
  name.textContent = item.name;
  const qty = document.createElement('span');
  qty.className = 'qty';
  qty.textContent = item.qty;
  li.append(name, qty);
  return li;
}
```

flour

2

ARE THE FOUR OUTPUTS THE SAME?

all four produced identical markup

TIMES THE ITEM'S NAME EXECUTED

0

The hostile name is an onerror attribute on an image that cannot load. A `<script>` tag inserted as markup does not run, which is exactly why it is the

COMPOSITION

WAY	SOURCE LINES	THE ATTACKER'S CODE RAN
string concatenation	14	no

► **PRESS "HOSTILE NAME" FIRST.** Two of the four execute it. Which two?

SIX LINES, AND THEY ARE THE WHOLE PRICE

```
1 function esc(value) {  
2   return String(value)  
3     .replaceAll('&', '&amp;')  
4     .replaceAll('<', '&lt;')  
5     .replaceAll('>', '&gt;')  
6     .replaceAll('"', '&quot;');  
7 }
```

► & **FIRST**, or you escape your own escapes. And `String(value)` first, because a number has no `replaceAll`.

WHEN TO USE WHICH, AND IT IS NOT A MATTER OF TASTE

TEMPLATE LITERAL + `esc`

A component you want to READ.
Shape is visible; the whole row fits on the screen.

`createElement`

A row that needs an element reference, a `dataset`, or a listener attached to it directly.

EITHER ONE WITHOUT `esc`

An item name is a string a user typed. It reaches the HTML parser, and it is the shortest wrong answer in the week.

► This assignment uses `createElement`, because your rows carry a `data-id` and the act after this one is about that.

WHERE A ROW SITS IS NOT WHAT A ROW IS

POSITION

```
1 li.dataset.index = index;  
2 // ...  
3 const i = Number(row.dataset.index);  
4 items[i].n += 1;
```

Correct until the first sort. Then it changes the wrong item and shows you the right one changing.

IDENTITY

```
1 li.dataset.id = item.id;  
2 // ...  
3 const id = row.dataset.id;  
4 setState({  
5   items: items.map((i) =>  
6     i.id === id ? { ...i, n: i.n + 1 } : i),  
7 });
```

Correct in every order, because the id does not move.

NOTHING IS WRONG UNTIL SOMETHING REORDERS THE LIST

the same six items · left keyed on position · right keyed on id reload open

TWO LISTS, ONE KEYED ON WHERE A ROW SITS AND ONE ON WHAT IT IS

sort both by quantity search both for "il" clear the search **+1 on the third row of each** remove the third row of each redraw both reset

KEYED ON POSITION — DATA-INDEX

flour	2
olive oil	1
almond milk	4
cheese	1
paprika	3
oat milk	2

nothing pressed yet

```
function bumpByIndex(row) {  
  /* The row's position in the list ON SCREEN, used as a p  
    COLLECTION. Those are the same number only while noth  
  const item = A.items[Number(row.dataset.index)];  
  if (!item) return null;  
  item.qty += 1;  
  row.querySelector('.qty').textContent = item.qty;  
  return item;  
}
```

KEYED ON IDENTITY — DATA-ID

flour	2
olive oil	1
almond milk	4
cheese	1
paprika	3
oat milk	2

nothing pressed yet

```
function bumpById(row) {  
  const item = B.items.find((candidate) => candidate.id ==  
  if (!item) return null;  
  item.qty += 1;  
  row.querySelector('.qty').textContent = item.qty;  
  return item;  
}
```

TIMES THE WRONG ITEM CHANGED

0

the two collections agree

Both lists start identical and stay identical while nothing reorders them. Sort first, then press a row button, and count.

► **PRESS +1 BEFORE YOU SORT.** The counter stays at zero. Now sort, and press it again.

A NEW ARRAY, EVERY TIME

```
1  setState({ items: [...items, newItem] });           // add
2  setState({ items: items.filter((i) => i.id !== id) }); // remove
3  setState({ items: items.map((i) => (i.id === id ? { ...i, n: i.n + 1 } : i)) }); // change
```

► `push` works today. It mutates the array **ALREADY INSIDE STATE**, so `setState` is handed the object it already had — and everything that ever compares old to new stops being able to.

ONE LISTENER, ON SOMETHING RENDER NEVER REPLACES

```
1 list.addEventListener('click', (event) => {  
2   const row = event.target.closest('li[data-id]');  
3   if (!row) return;  
4   // ... setState from row.dataset.id  
5 });
```

► Week 8 made delegation a good habit. **THIS WEEK IT STOPS BEING OPTIONAL:** before today you chose when to redraw; from today it happens on every keystroke.

SEARCH, FILTER AND SORT ARE NOT THREE BEHAVIOURS

```
1 export function selectVisible(state) {  
2   return state.items  
3     .filter((item) => item.title.includes(state.query))  
4     .filter((item) => state.category === 'all' || item.category === state.category)  
5     .sort((a, b) =>  
6       state.sort === 'number' ? a.number - b.number : a.title.localeCompare(b.title, 'he'),  
7     );  
8 }
```

They are one derivation, and
the list on screen is its output.



THEN THE THREE HANDLERS ARE ONE LINE EACH

```
1 queryField.addEventListener('input', () => setState({ query: queryField.value }));  
2 categorySelect.addEventListener('change', () => setState({ category: categorySelect.value }));  
3 sortSelect.addEventListener('change', () => setState({ sort: sortSelect.value }));
```

► Note what the third one does **NOT** do: it does not sort anything. A sort handler longer than one line is sorting, and sorting is a derivation.

HOW LONG A STORED ANSWER STAYS TRUE

four questions about one collection · stored on the left, derived on the right↺ reloadopen ↗

FOUR QUESTIONS ABOUT ONE COLLECTION, STORED ON THE LEFT AND DERIVED ON THE RIGHT

add an item · updates everything

remove one · updates everything

quick fix: bulk import · updates the collection only

rename a shelf · updates the collection only

reset

STORED — FOUR FIELDS BESIDE THE COLLECTION

QUESTION	THE STORED ANSWER	ASKED AFRESH
count	3	3
total	7	7
shelves	cold, dry	cold, dry
heaviest	almond milk	almond milk

nothing changed yet

DERIVED — FOUR FUNCTIONS, NO FIELDS

count(items)3

total(items)7

shelves(items)cold, dry

heaviest(items)almond milk

Nothing here can go out of date, and the reason is not discipline. There is nothing here to go out of date.

STORED ANSWERS CURRENTLY LYING

0

the two panels agree

answering all four afresh, averaged over 1000 runs: 0.0006 ms. Drawing them once: 0.100 ms. Act 9 is where those numbers start to matter.

► The first two buttons keep every stored field correct.

THE THIRD ONE IS WRITTEN A MONTH LATER, BY SOMEBODY WHO ONLY NEEDED TO CHANGE THE COLLECTION.

THE EMPTY STATE IS A PROPERTY OF THE DATA, AND THERE ARE TWO OF THEM

```
1  const visible = selectVisible(state);  
2  empty.hidden = visible.length > 0;  
3  empty.textContent =  
4    state.items.length === 0  
5    ? 'nothing here yet – add your first item'  
6    : 'nothing matches the current search and filter';
```

► “There is nothing here” sends the user to the add form. “Nothing matched” sends them to the search box.

ONE SENTENCE FOR BOTH IS A WRONG SIGN.

THE THIRD PAGE HAS NO DATA, AND THAT IS THE PROOF

```
1 export function render(state) {  
2   renderList(state);    // returns immediately if #list is not on this page  
3   renderDetail(state);  
4   renderSummary(state); // returns immediately if #summary-count is not here  
5 }
```

► One entry point, three pages, and no `if` anywhere that asks which page it is on. **ADDING A FOURTH PAGE IS A MOUNT AND A FUNCTION.**

LIVE CODING · 5 MINUTES

ADD A FILTER WHOSE OPTIONS NOBODY WROTE

- 1 Add `selectCategories(state)` to `state.js`: a `Set` over the collection, sorted.
- 2 In `render`, rebuild the `<option>`s from it — keeping `value` before and after.
- 3 Delete the last item of a category **and watch its option disappear**. Nobody wrote that line.

► Step 2 hides the only exception in the whole week, and it is worth naming out loud.

WHY THE SPLIT MAKES EVERY FUTURE CHANGE LOCAL

the same 47 lines • as one file and as three • pick a change request

reload

open

THE SAME 46 LINES, AS ONE FILE AND AS THREE, AND WHERE A CHANGE LANDS

change request

ONE FILE — APP.JS

9 to change • 47 to read

one file, so there is no line you can be sure the change does not touch.
app.js

```
1 import { SEED } from './items.js';
2
3 let items = SEED.map((item) => ({ ...item }));
4 let query = '';
5 let sort = 'name';
6
7 const list = document.querySelector('#list');
8 const count = document.querySelector('#count');
9 const queryField = document.querySelector('#query');
10 const sortField = document.querySelector('#sort');
11 const form = document.querySelector('#item-form');
12
13 function visible() {
14   return items
15     .filter((item) => item.title.includes(query))
```

THREE FILES

10 to change • 19 to read

a change to how a row LOOKS. It has one home. One file opened.
state.js

```
1 let state = { items: [], query: '', sort: 'name' };
2 const listeners = new Set();
3
4 export const getState = () => state;
5
6 export function setState(patch) {
7   state = { ...state, ...patch };
8   for (const listener of listeners) listener(state);
9 }
10
11 export function selectVisible(state) {
12   return state.items
13     .filter((item) => item.title.includes(state.query))
14     .sort((a, b) =>
15       state.sort === 'number'
```

► Do not watch the “lines to change” number — it is nearly the same. Watch **LINES TO READ**.

PERFORMANCE, HONESTLY

10 / 100 / 1000 / 5000 rows · measured on this machine, now ↻ reload open ↗

WHAT A FULL REDRAW COSTS, AND WHAT ACTUALLY BREAKS FIRST

measure at 10 / 100 / 1000 / 5000 rows | list size 100 ▾ | redraw this list | redraw narrowed to 5 rows | patch one row

TIMINGS, ON THIS MACHINE, JUST NOW

ROWS	FULL REDRAW	REDRAW + LAYOUT	PATCH ONE ROW
not measured yet			

One frame at 60 Hz is 16.7 ms. Anything under it is invisible.

THE LIST

item 0

0

item 1

1

item 2

2

item 3

3

The text box is INSIDE the list and it redraws on every keystroke, which is exactly what a search box does. Type one character into it.

0

redraws that destroyed the focused element

0

redraws that threw away the scroll position. Scroll down first, then try each of the two redraw buttons — only one of them costs you the place you were.

WHERE THIS STOPS BEING FINE

a few hundred rows	fine
a control inside the redrawn region	not fine
a redraw on every keystroke over thousands of rows	not fine
an animation or a video inside the region	not fine

Only the first of those four is about speed. The fix for the other three is the same and it is not a framework: keep anything the user is holding on to OUTSIDE the part you replace.

► One frame at 60 Hz is 16.7 ms. **A FEW HUNDRED ROWS IS COMFORTABLY INSIDE IT**, and these numbers are this laptop's, not a blog post's.

WHERE A FULL REDRAW STOPS BEING FINE

SITUATION	FINE?	THE FIX, AND IT IS NOT A FRAMEWORK
a few hundred rows	yes	—
a control the user is holding, inside the region	no	move the control OUT of the region
a redraw per keystroke over thousands of rows	no	debounce, or render a window of the list
an animation or a video inside the region	no	move it out, or replace only what changed

Only the first row is about speed.



THE FIVE MISTAKES, AND WHAT EACH ONE COSTS

THE MISTAKE	HOW IT SHOWS UP
update the DOM and the array separately	a removed item comes back on the next add
store something derivable	works perfectly until somebody changes the collection elsewhere
use the array index as identity	changes the wrong item, and shows the right one changing
a handler that draws	three things to remember per handler, forever
one long file with no layers	every change means reading all of it

REVIEW THE MACHINE · 4 MINUTES

CLAUDE REFACTORED IT. FIND THE TWO FAULTS.

```
1 function initialState() {  
2   return { items: [], query: '', visibleItems: [] };  
3 }  
4  
5 export function render(state) {  
6   const list = document.querySelector('#list');  
7   list.replaceChildren(...state.visibleItems.map(itemRow));  
8  
9   list.addEventListener('click', (event) => {  
10    const row = event.target.closest('li[data-id]');  
11    if (row) remove(row.dataset.id);  
12  });  
13 }
```

► Both are the shortest reasonable thing to write, both pass a manual test, and both are on today's rubric.

THREE QUESTIONS BEFORE YOU GO

- 1 Your state has `items` and `visibleItems`.
Which line of code will eventually be wrong, and who writes it?
- 2 You call `render(state)` twice in a row with the same object.
What must be true of the screen, and what does it prove?
- 3 A sort handler in your code is four lines long.
What is in the other three, and where do they belong?

THE MINI-SURGERY, AND WHY THE FORMAT MATTERS MORE THAN THE MARKS

TASK	THE WORDING, AND IT IS WEEK 13'S	MINUTES
1 · repair	"The application does X. It should do Y. Find the cause and repair it."	10
2 · extend	"Add Z. Change no behaviour that already works."	10
3 · restructure	"Move this so that a future change to it is local. Behaviour identical."	10

► Same three verbs in week 13, on **YOUR OWN** project, at three times the size. Today is a rehearsal, and it is **GRADED LIGHTLY ON PURPOSE**.

TODAY'S HANDS-ON, AND IT IS IN TWO HALVES

30 MINUTES · ALONE

Mini-surgery. One small debug, one small feature, one small refactor, on an application you have never seen. Same wording as week 13, one third the size. Graded lightly — the point is calibration.

60 MINUTES · YOUR PROJECT

Refactor it to the one-way loop.
This is a hard requirement: the rubric and the auto-grader both check for a single state object, a render function, and handlers that do not draw.

STRAIGHT AFTERWARDS

The mini-surgery walkthrough, in public, on the projector. **Bring the sheet you filled in.**

► Next week the collection stops living in memory: `JSON`, `localStorage`, and a page that survives a refresh.